

LAB ASSIGNMENT 10

Problem Statement 1

Design and implement a parallel program using the **Message Passing Interface (MPI)** to demonstrate synchronous point-to-point communication between two processes. The program should perform a "message swap" or a specific data exchange to illustrate how buffers are sent and received across different process ranks.

Objectives

- Initialize the MPI environment and determine the total number of processes and individual process ranks.
- Implement a safety check to ensure the program only executes when exactly **two processes** are active.
- Dynamically allocate memory for large message buffers (e.g., 1 MB arrays).
- Utilize MPI_Send and MPI_Recv to exchange data between Rank 0 and Rank 1.
- Verify data integrity by printing the values of the received buffers.

```
#include <mpi.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

int main(int argc, char *argv[])

{

    MPI_Init(&argc, &argv);

    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2)

    {
```

```

        if (rank == 0)
        {
            printf("Error: This program requires exactly 2
processes.\n");
        }

        MPI_Finalize();

        return 0;
    }

    const int BUFFER_SIZE = 1024 * 1024; // 1 MB

    char *send_buffer = (char *)malloc(BUFFER_SIZE * sizeof(char));
    char *recv_buffer = (char *)malloc(BUFFER_SIZE * sizeof(char));

    if (rank == 0)
        memset(send_buffer, 'A', BUFFER_SIZE);

    // Message Swap

    if (rank == 0)
    {
        printf("0 is sending\n");

        MPI_Send(send_buffer, 1048576, MPI_CHAR, 1, 0, MPI_COMM_WORLD);

        printf("0 sent\n");

        // MPI_Recv(recv_buffer, BUFFER_SIZE, MPI_CHAR, 1, 0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        printf("2\n");
    }

    else if (rank == 1)
    {
        printf("1 is ready to recieve\n");
    }
}

```

```
    MPI_Recv(recv_buffer, 1048576, MPI_CHAR, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    printf("1 recieved\n");

    // MPI_Send(send_buffer, BUFFER_SIZE, MPI_CHAR, 0, 0,
MPI_COMM_WORLD);

    printf("Rank %d received: ", rank);

    for (int i = 0; i < 10; i++)
    {
        printf("%c", recv_buffer[i]);
    }

    printf("1\n");
}

free(send_buffer);

free(recv_buffer);

MPI_Finalize();
}
```

```

[adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~$] [ 3] /usr/lib/x86_64-linux-gnu/openmpi/lib/openmpi3/mca_pml_ob1.so(mca_pml_ob1_send_request_schedule_once+0x1c6)[0x7f8488af6926]
[adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~$] [ 4] /usr/lib/x86_64-linux-gnu/openmpi/lib/openmpi3/mca_pml_ob1.so(mca_pml_ob1_recv_frag_callback_ack+0x1a9)[0x7f8488aef429]
[adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~$] [ 5] /usr/lib/x86_64-linux-gnu/openmpi/lib/openmpi3/mca_btl_vader.so(mca_btl_vader_poll_handle_frag+0x95)[0x7f84890deds]
[adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~$] [ 6] /usr/lib/x86_64-linux-gnu/openmpi/lib/openmpi3/mca_btl_vader.so(+0x51a5)[0x7f84890de1a5]
[adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~$] [ 7] /lib/x86_64-linux-gnu/libopen-pal.so.40(opal_progress+0x34)[0x7f848ad97854]
[adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~$] [ 8] /usr/lib/x86_64-linux-gnu/openmpi/lib/openmpi3/mca_pml_ob1.so(mca_pml_ob1_send+0x7c5)[0x7f8488aee5f5]
[adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~$] [ 9] /lib/x86_64-linux-gnu/libmpi.so.40(PMPP_send+0x123)[0x7f848b12593]
[adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~$] [10] 1(+0x13d2)[0x55cf657783d2]
[adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~$] [11] /lib/x86_64-linux-gnu/libc.so.6(__libc_start_main+0xf3)[0x7f848aefb083]
[adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~$] [12] 1(+0x13d2)[0x55cf657783d2]
*** End of error message ***
Primary job terminated normally, but 1 process returned
a non-zero exit code. Per user-direction, the job has been aborted.
-----
mpirun noticed that process rank 0 with PID 0 on node administrator-hp-prodesk-400-g7-small-form-factor-pc exited on signal 11 (Segmentation fault).
-----
(base) adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~/Desktop/U23AI052/10$ mpirun 1.c -o 1
1 is ready to receive
0 is sending
0 sent
2
1 received
Rank 1 received: AAAAAAAAAA
(base) adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~/Desktop/U23AI052/10$ mpirun -np 2 1
1 is ready to receive
0 is sending
1 received
Rank 1 received: AAAAAAAAAA
0 sent
2
1 received
Rank 1 received: AAAAAAAAAA
(base) adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~/Desktop/U23AI052/10$ mpirun -np 2 1
1 is ready to receive
0 is sending
0 sent
2
1 received
Rank 1 received: AAAAAAAAAA
(base) adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~/Desktop/U23AI052/10$ mpirun -np 2 1
1 is ready to receive
0 is sending
0 sent
2
1 received
Rank 1 received: AAAAAAAAAA
(base) adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~/Desktop/U23AI052/10$ mpirun -np 2 1
1 is ready to receive
0 is sending
0 sent
2
1 received
Rank 1 received: AAAAAAAAAA
(base) adminstrator@hp-prodesk-400-g7-small-form-factor-pc:~/Desktop/U23AI052/10$

```

As processes are running in parallel so either rank 1 or either rank 2 starts working first but send and receive are blocking calls. Also the data transfer is taking place perfectly.

Problem Statement 2

In MPI communication, the sender and the receiver must agree on the **count** and the **datatype** of the message being exchanged. If there is a discrepancy between the MPI datatype handle used in `MPI_Send` and the one used in `MPI_Recv`, the resulting data will often be corrupted due to differences in memory allocation and bit-representation.

Part A: Demonstrating the Mismatch

Write an MPI program that implements the mismatch scenario. Your code must:

1. Initialize a double on Rank 0 and a float on Rank 1. You can take any distinct datatype on send and receive.
2. Perform a point-to-point communication (`MPI_Send` / `MPI_Recv`) using mismatched MPI handles.
3. Print the values on both ranks to demonstrate that the data received by Rank 1 is incorrect.

```

#include <mpi.h>

#include <stdio.h>

```

```
int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2)
    {
        if (rank == 0)
            printf("Run with exactly 2 processes\n");
        MPI_Finalize();
        return 0;
    }

    if (rank == 0)
    {
        double send_val = 12345.6789; // 8 bytes
        printf("Rank 0 sending double: %lf\n", send_val);

        MPI_Send(&send_val, 1, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD);
    }

    else if (rank == 1)
    {
        float recv_val[2];

        MPI_Recv(recv_val, 2, MPI_FLOAT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
```

```

printf("Rank 1 received values (corrupted): %f %f\n",
recv_val[0], recv_val[1]);

}

MPI_Finalize();
}

```

```

(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc:~/Desktop/U23AI052/10$ mpicc 2.c -o 2
Rank 0 sending double: 12345.678900
[administrator-hp-prodesk-400-g7-small-form-factor-pc:08865] *** An error occurred in MPI_Recv
[administrator-hp-prodesk-400-g7-small-form-factor-pc:08865] *** reported by process [3578855425,1]
[administrator-hp-prodesk-400-g7-small-form-factor-pc:08865] *** on communicator MPI_COMM_WORLD
[administrator-hp-prodesk-400-g7-small-form-factor-pc:08865] *** MPI_ERR_TRUNCATE: message truncated
[administrator-hp-prodesk-400-g7-small-form-factor-pc:08865] *** MPI_ERRORS_ARE_FATAL (processes in this communicator will now abort,
[administrator-hp-prodesk-400-g7-small-form-factor-pc:08865] *** and potentially your MPI job)
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc:~/Desktop/U23AI052/10$ ^C
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc:~/Desktop/U23AI052/10$ mpicc 2.c -o 2
Rank 0 sending double: 12345.678900
Rank 1 received values (corrupted): -21011315317711819505664.000000 6.253520
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc:~/Desktop/U23AI052/10$

```

The main observation is that if i sent double to single float it gave abrupt error but to check data is correct we used 2 floats as size of 2 floats = 1 double.

Part B: Identifying and Implementing the Solution

After observing the failure, you must provide a solution. Modify your program to ensure data integrity. Consider the following in your fix:

- How must the variable declarations (C types) relate to the MPI handles?
- What happens if you change the MPI handle without changing the underlying C variable type?
- Rather than sending same datatype on send receive you can send same datatype and size combo to be same. Check this produces the correct result.
- Ensure the final output shows Rank 1 successfully receiving the value 1.000000.

```

#include <mpi.h>

#include <stdio.h>

```

```
int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2)
    {
        if (rank == 0)
            printf("Run with exactly 2 processes\n");
        MPI_Finalize();
        return 0;
    }

    if (rank == 0)
    {
        float send_val = 1.0f;
        printf("Rank 0 sending: %f\n", send_val);

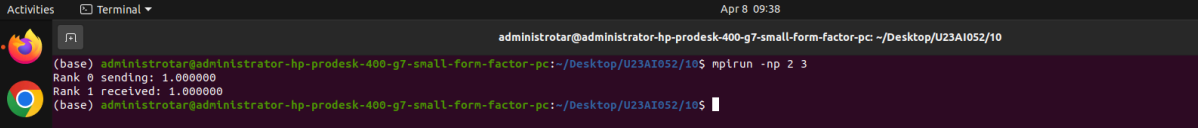
        MPI_Send(&send_val, 1, MPI_FLOAT, 1, 0, MPI_COMM_WORLD);
    }

    else if (rank == 1)
    {
        float recv_val;
```

```
MPI_Recv(&recv_val, 1, MPI_FLOAT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

printf("Rank 1 received: %f\n", recv_val);
}

MPI_Finalize();
}
```



Even if types differ, communication works **ONLY IF** Total bytes match

Problem Statement 3

Implement a C program using the **Message Passing Interface (MPI)** to demonstrate synchronous point-to-point communication between two processes. The program must fulfill the following requirements:

1. **Process 0 Operations:**

- o Initialize an integer array of size 10 or any with values representing their index ($\text{num}[i] = i$).
- o Send the array to **Process 1** using **MPI_Send**.
- o Receive a modified array back from **Process 1** into a separate buffer using **MPI_Recv**.
- o Display the received values.

2. **Process 1 Operations:**

- o Receive the integer array from **Process 0** and print its contents.
- o Modify the array elements (e.g., reversing the order or applying a transformation).
- o Send the modified array back to **Process 0**.

```
#include <mpi.h>

#include <stdio.h>

#include <unistd.h> // for sleep()
```

```
#define N 10

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int arr[N];

    if (rank == 0)
    {
        // Initialize array
        for (int i = 0; i < N; i++)
            arr[i] = i;

        printf("Rank 0 sending array to Rank 1\n");

        MPI_Send(arr, N, MPI_INT, 1, 0, MPI_COMM_WORLD);

        MPI_Recv(arr, N, MPI_INT, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

        printf("Rank 0 received modified array:\n");
        for (int i = 0; i < N; i++)
            printf("%d ", arr[i]);
    }
}
```

```
        printf("\n");

    }

    else if (rank == 1)

    {

        MPI_Recv(arr, N, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

        printf("Rank 1 received array:\n");

        for (int i = 0; i < N; i++)

            printf("%d ", arr[i]);

        printf("\n");

        // reverse

        for (int i = 0; i < N / 2; i++)

        {

            int temp = arr[i];

            arr[i] = arr[N - i - 1];

            arr[N - i - 1] = temp;

        }

        printf("Rank 1 sending modified array back\n");

        MPI_Send(arr, N, MPI_INT, 0, 0, MPI_COMM_WORLD);

    }

    else

    {

        // Other ranks doing independent work

        for (int i = 0; i < 5; i++)

        {
```

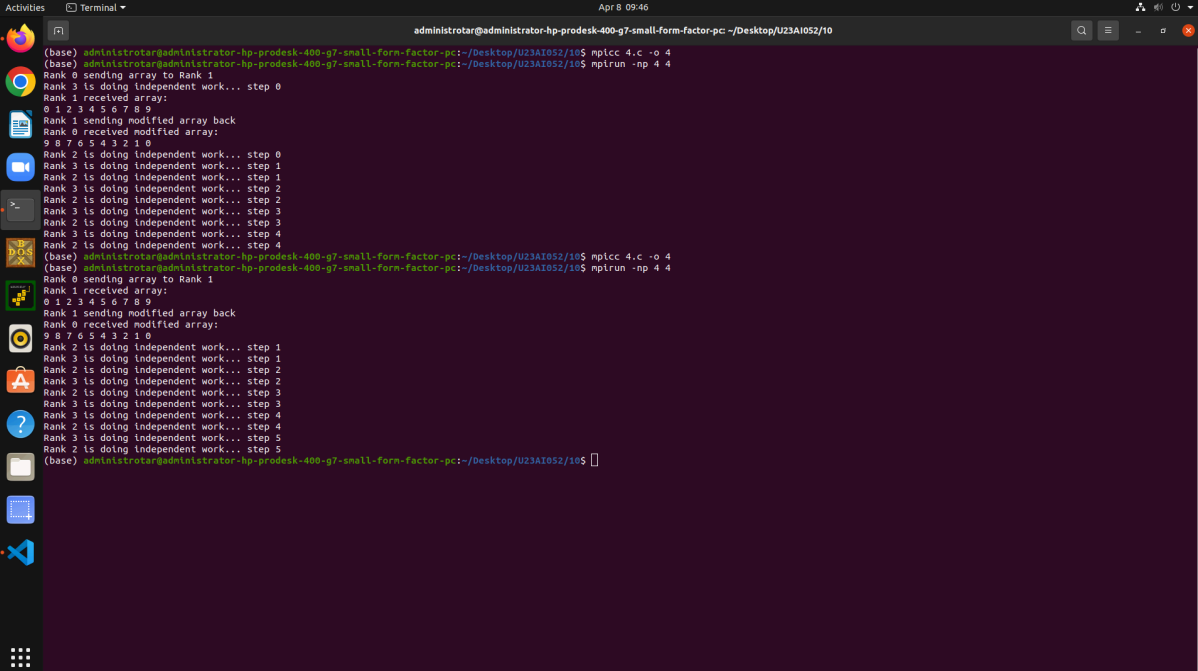
```
printf("Rank %d is doing independent work... step %d\n",
rank, i);

sleep(1); // simulate work

}

}

MPI_Finalize();
}
```



While Rank 0 and Rank 1 are engaged in point-to-point communication, the remaining ranks execute independently in parallel, demonstrating that MPI processes are isolated unless explicitly involved in communication.

Problem Statement 4

In distributed computing using the **Message Passing Interface (MPI)**, processes often need to exchange data. A common scenario involves two processes—let's call them **Rank 0** and **Rank 1**—where both need to send a large buffer of data to each other simultaneously.

In this exercise, you are required to simulate a scenario where:

1. Both processes initialize two large integer arrays (e.g., 10,000 elements).
2. A synchronization point (Barrier) is established to ensure both processes start communication at the same time.
3. **Rank 0** attempts to **Send** its data to Rank 1 and then **Receive** data from Rank 1.

4. **Rank 1** attempts to **Send** its data to Rank 0 and then **Receive** data from Rank 0.

The Challenge

When using standard blocking calls (like `MPI_Send` and `MPI_Recv`), the program may "hang" or freeze indefinitely. This occurs because `MPI_Send` may wait for a matching `MPI_Recv` to be initiated on the destination process before it finishes. If both processes are stuck waiting for their Send to complete, neither will ever reach the Recv instruction.

Fix the problem using `MPI_Isend` and `MPI_Irecv`.

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N 10000

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int *send_buf = (int *)malloc(N * sizeof(int));
    int *recv_buf = (int *)malloc(N * sizeof(int));

    for (int i = 0; i < N; i++)
        send_buf[i] = rank * 100 + i;

    MPI_Barrier(MPI_COMM_WORLD);
    double start_block = MPI_Wtime();

    if (rank == 0)
```

```

{
    MPI_Send(send_buf, N, MPI_INT, 1, 0, MPI_COMM_WORLD);

    MPI_Recv(recv_buf, N, MPI_INT, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
}

else

{
    MPI_Recv(recv_buf, N, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    MPI_Send(send_buf, N, MPI_INT, 0, 0, MPI_COMM_WORLD);
}

double end_block = MPI_Wtime();

MPI_Barrier(MPI_COMM_WORLD);

double start_nonblock = MPI_Wtime();

MPI_Request reqs[2];

    MPI_Isend(send_buf, N, MPI_INT, 1 - rank, 0, MPI_COMM_WORLD,
&reqs[0]);

    MPI_Irecv(recv_buf, N, MPI_INT, 1 - rank, 0, MPI_COMM_WORLD,
&reqs[1]);

    MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);

double end_nonblock = MPI_Wtime();

if (rank == 0)

{

```

```

printf("Blocking time    : %f seconds\n", end_block -
start_block);

printf("Non-blocking time: %f seconds\n", end_nonblock -
start_nonblock);

}

free(send_buf);

free(recv_buf);

MPI_Finalize();

return 0;

}

```

```

0 1 2 3 4 5 6 7 8 9
Rank 1 sending modified array back
Rank 0 received modified array:
9 8 7 6 5 4 3 2 1 0
Rank 2 is doing independent work... step 0
Rank 3 is doing independent work... step 1
Rank 2 is doing independent work... step 1
Rank 3 is doing independent work... step 2
Rank 2 is doing independent work... step 2
Rank 3 is doing independent work... step 3
Rank 2 is doing independent work... step 3
Rank 3 is doing independent work... step 4
Rank 2 is doing independent work... step 4
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpicc 4.c -o 4
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 4 4
Rank 0 sending array to Rank 1
Rank 1 received array:
0 1 2 3 4 5 6 7 8 9
Rank 1 sending modified array back
Rank 0 received modified array:
9 8 7 6 5 4 3 2 1 0
Rank 2 is doing independent work... step 1
Rank 3 is doing independent work... step 1
Rank 2 is doing independent work... step 2
Rank 3 is doing independent work... step 2
Rank 2 is doing independent work... step 3
Rank 3 is doing independent work... step 3
Rank 2 is doing independent work... step 4
Rank 3 is doing independent work... step 4
Rank 2 is doing independent work... step 5
Rank 3 is doing independent work... step 5
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpicc 5.c -o 5
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 4 5
Run with exactly 2 processes
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 4 2
Run with exactly 2 processes
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 2 4
Rank 1 received array:
0 1 2 3 4 5 6 7 8 9
Rank 1 sending modified array back
Rank 0 sending array to Rank 1
Rank 0 received modified array:
9 8 7 6 5 4 3 2 1 0
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 2 5
Rank 0 received first 5 values: 100 101 102 103 104
Rank 0 communication time: 0.000024 seconds
Rank 1 received first 5 values: 0 1 2 3 4
Rank 1 communication time: 0.000021 seconds
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpicc 5.c -o 5
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 2 5
--- Timing Comparison ---
Blocking time    : 0.000037 seconds
Non-blocking time: 0.000007 seconds
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$

```

Isend and Irecvie is faster

```

#include <mpi.h>

#include <stdio.h>

#include <stdlib.h>

#define N 10000

```

```
int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2)
    {
        if (rank == 0)
            printf("Run with exactly 2 processes\n");
        MPI_Finalize();
        return 0;
    }

    int *send_buf = (int *)malloc(N * sizeof(int));
    int *recv_buf = (int *)malloc(N * sizeof(int));

    for (int i = 0; i < N; i++)
        send_buf[i] = rank * 100 + i;

    if (rank == 0)
    {
        printf("Rank 0 trying to send...\n");

        MPI_Send(send_buf, N, MPI_INT, 1, 0, MPI_COMM_WORLD);

        printf("Rank 0 finished send\n");
    }
}
```

```

        MPI_Recv(recv_buf, N, MPI_INT, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    }

    else if (rank == 1)

    {

        printf("Rank 1 trying to send...\n");

        MPI_Send(send_buf, N, MPI_INT, 0, 0, MPI_COMM_WORLD);

        printf("Rank 1 finished send\n");

        MPI_Recv(recv_buf, N, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    }

    free(send_buf);

    free(recv_buf);

    MPI_Finalize();

}

```

The screenshot shows a terminal window with the following output:

```

Rank 2 is doing independent work... step 0
Rank 3 is doing independent work... step 1
Rank 2 is doing independent work... step 1
Rank 3 is doing independent work... step 2
Rank 2 is doing independent work... step 2
Rank 3 is doing independent work... step 3
Rank 2 is doing independent work... step 3
Rank 3 is doing independent work... step 4
Rank 2 is doing independent work... step 4
Rank 3 is doing independent work... step 5
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpicc 4.c -o 4
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 4 4
Rank 0 sending array to Rank 1
Rank 1 received array:
0 1 2 3 4 5 6 7 8 9
Rank 1 sending modified array back
Rank 0 received modified array:
9 8 7 6 5 4 3 2 1 0
Rank 2 is doing independent work... step 1
Rank 3 is doing independent work... step 1
Rank 2 is doing independent work... step 2
Rank 3 is doing independent work... step 2
Rank 2 is doing independent work... step 3
Rank 3 is doing independent work... step 3
Rank 2 is doing independent work... step 4
Rank 3 is doing independent work... step 4
Rank 2 is doing independent work... step 5
Rank 3 is doing independent work... step 5
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpicc 5.c -o 5
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 4 5
Run with exactly 2 processes
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 4 2
Run with exactly 2 processes
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 2 4
Rank 1 received array:
0 1 2 3 4 5 6 7 8 9
Rank 1 sending modified array back
Rank 0 received modified array:
9 8 7 6 5 4 3 2 1 0
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 2 5
Rank 0 received first 5 values: 100 101 102 103 104
Rank 0 communication time: 0.000024 seconds
Rank 1 received first 5 values: 0 1 2 3 4
Rank 1 communication time: 0.000021 seconds
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpicc 6.c -o 6
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 2 6
--- Timing Comparison
Blocking time: 0.000027 seconds
Non-blocking time: 0.000007 seconds
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpicc 6.c -o 6
(base) administrator@administrator-hp-prodesk-400-g7-small-form-factor-pc: ~/Desktop/U23AI052/10$ mpirun -np 2 6
Rank 1 trying to send...
Rank 0 trying to send...

```

